

## Exercise 2: E-commerce Platform Search Function

### Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

### Steps:

#### 1. Understand Asymptotic Notation:

- Explain Big O notation and how it helps in analyzing algorithms.
- Describe the best, average, and worst-case scenarios for search operations.

#### 2. Setup:

- Create a class of **Product** with attributes for searching, such as **productId**, **productName**, and **category**.

#### 3. Implementation:

- Implement linear search and binary search algorithms.
- Store products in an array for linear search and a sorted array for binary search.

#### 4. Analysis:

- Compare the time complexity of linear and binary search algorithms.
- Discuss which algorithm is more suitable for your platform and why.

Answer:

What is Big O Notation?

Big O notation is a way of measuring how the running time or memory usage of an algorithm grows as the size of the input grows .

It enables us:

Compare algorithms

Predict performance on large data sets.

Choose the most effective algorithm.

Typical Time Complexities

Big O What it means Example

Constant Time  $O(1)$  .Array element retrieval

$O(\log n)$  Logarithmic Time Binary Search  $O(n)$  Linear Time Linear Search  $O(n^2)$  Quadratic Time Nested Loops  $O(2^n)$  Exponential Time Recursive Fibonacci

Linear Search: Best, Average, and Worst Case

Best Case:  $O(1)$

Element is present in the 1st position.

Average Case:  $O(n)$  Element is somewhere in the middle.

Worst Case:  $O(n)$  Element at last position or not present.

Binary Search

- **Best Case:**  $O(1)$   
Element is exactly in the middle.
- **Average Case:**  $O(\log n)$
- **Worst Case:**  $O(\log n)$

Code:

```
public class SearchDemo {  
    // Product Class  
    static class Product {  
        int productId;  
        String productName;  
        String category;  
        Product(int productId, String productName, String category) {  
            this.productId = productId;  
            this.productName = productName;  
            this.category = category;  
        }  
        void display() {  
            System.out.println("Product ID : " + productId);  
            System.out.println("Product Name : " + productName);  
            System.out.println("Category : " + category);  
        }  
    }  
}
```

*// Linear Search*

```
public static Product linearSearch(Product[] products, int id) {  
    for (int i = 0; i < products.length; i++) {  
        if (products[i].productId == id) {  
            return products[i];  
        }  
    }  
    return null;  
}
```

*// Binary Search*

```
public static Product binarySearch(Product[] products, int id) {  
    int low = 0;  
    int high = products.length - 1;  
    while (low <= high) {  
        int mid = (low + high) / 2;  
        if (products[mid].productId == id) {  
            return products[mid];  
        }  
        if (products[mid].productId < id) {  
            low = mid + 1;  
        } else {  
            high = mid - 1;  
        }  
    }  
    return null;  
}
```

*// Main Method*

```
public static void main(String[] args) {  
    Product[] products = {  
        new Product(101, "Laptop", "Electronics"),  
        new Product(102, "Shoes", "Fashion"),  
    }  
}
```

```

        new Product(103, "Mobile", "Electronics"),
        new Product(104, "Watch", "Accessories"),
        new Product(105, "Bag", "Fashion")
    };
    System.out.println("---- Linear Search----");
    Product result1 = linearSearch(products, 104);
    if (result1 != null) {
        result1.display();
    } else {
        System.out.println("Product not found.");
    }
    System.out.println();
    System.out.println("---- Binary Search----");
    Product result2 = binarySearch(products, 104);
    if (result2 != null) {
        result2.display();
    } else {
        System.out.println("Product not found.");
    }
}
}

```

Output:

```

PS C:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE\Design Patterns and Principles
> cd "c:\Users\upend\Downloads\Cognizant GITHUB\Cognizant-JavaFSE\Algorithms_Data Structures\
Category : Accessories

---- Binary Search ----
Product ID : 104
Product Name : Watch
Category : Accessories

```

Analysis:

## Which Algorithm is More Suitable?

### Linear Search

- Works on both sorted and unsorted arrays.
- Easy to understand and implement.
- Good for small amounts of data.

### Binary Search

- Works only when the array is sorted.
- Finds the product much faster.
- Checks fewer elements, so it saves time.

### Conclusion

For an e-commerce platform, **Binary Search** is a better choice because online shopping websites usually have a large number of products. If the products are stored in sorted order (for example, by **productId**), Binary Search can find a product very quickly by reducing the search area in every step. This makes it faster than Linear Search, which checks each product one by one. Therefore, Binary Search is more efficient for large datasets and provides better search performance.

## Exercise 7: Financial Forecasting

### Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

### Steps:

1. **Understand Recursive Algorithms:**
  - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
  - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
  - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**

- Discuss the complexity of your recursive algorithm.
- Explain how to optimize the recursive solution to avoid excessive computation.

Answer:

## What is Recursion?

Recursion is a programming technique where a method calls to solve a problem. It breaks a large problem into smaller and simpler parts until a stopping condition, called the **base case**, is reached. Recursion makes some problems easier to understand and write, especially those that involve repeated calculations or patterns.

Code:

```
public class FinancialForecast {
    public static double futureValue(double amount, double rate, int years) {
        if (years == 0) {
            return amount;
        }
        return futureValue(amount * (1 + rate), rate, years - 1);
    }
    public static void main(String[] args) {
        double principal = 10000;
        double growthRate = 0.10;
        int years = 5;
        double result = futureValue(principal, growthRate, years);
        System.out.println("Future Value = " + result);
    }
}
```

Output:

```
● PS C:\Users\upend\Downloads\Cognizant_GITHUB\Cognizant-JavaFSE\Algorithms_Data Structures> cd
"c:\Users\upend\Downloads\Cognizant_GITHUB\Cognizant-JavaFSE\Algorithms_Data Structures\" ;
if ($?) { javac FinancialForecast.java } ; if ($?) { java FinancialForecast }
Future Value = 16105.100000000008
```

## Step 4: Analysis

### Time Complexity

The recursive method makes one recursive call for each year.

- **Time Complexity:**  $O(n)$ , where  $n$  is the number of years.
- **Space Complexity:**  $O(n)$  because each recursive call is stored on the call stack.

### How to Optimize the Recursive Solution

The recursive method uses extra memory because each function call is stored.

An iterative loop is more efficient as it uses less memory while giving the same result.

For complex recursive problems, **memoization** can be used to save previous results and avoid repeated calculations.